

Programmation Orientée Objet

— UML —

Cédric Buche

École Nationale d'Ingénieurs de Brest (ENIB)

20 novembre 2013



Plan

- 1 Introduction
- 2 Diagramme de classes (4 UC)
- 3 Diagramme de cas d'utilisation (1 UC)
- 4 Diagrammes d'interaction (2 UC)

Plan

- 1 Introduction
- 2 Diagramme de classes (4 UC)
- 3 Diagramme de cas d'utilisation (1 UC)
- 4 Diagrammes d'interaction (2 UC)

Objectifs du cours – prérequis

Objectifs :

- Connaître le langage de modélisation UML
- Comprendre la sémantique des principaux éléments des différents modèles

Prérequis :

- Maîtriser les principes de la programmation orientée–objet

Importance de la modélisation

La niche, la maison familiale et l'immeuble



: quelques planches, des clous, un marteau et quelques outils.

: plans généraux, plans d'exécution détaillés (pièces, électricité, plomberie, chauffage)

: planification détaillée, nombreux plans et études

Importance de la modélisation

La niche, la maison familiale et l'immeuble



: quelques planches, des clous, un marteau et quelques outils.



: plans généraux, plans d'exécution détaillés (pièces, électricité, plomberie, chauffage)

: planification détaillée, nombreux plans et études

Importance de la modélisation

La niche, la maison familiale et l'immeuble



: quelques planches, des clous, un marteau et quelques outils.



: plans généraux, plans d'exécution détaillés (pièces, électricité, plomberie, chauffage)



: planification détaillée, nombreux plans et études

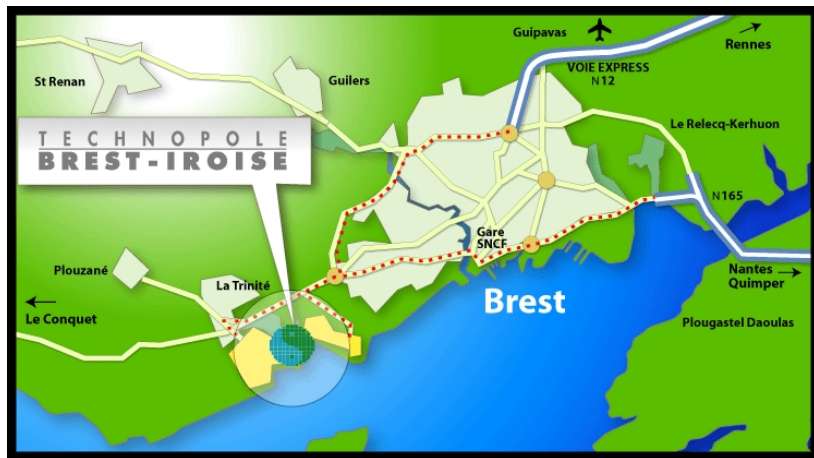
Pourquoi modéliser ?

- Mieux comprendre le système en développement
- Appréhender ces systèmes dans leur entièreté → modèles de systèmes complexes



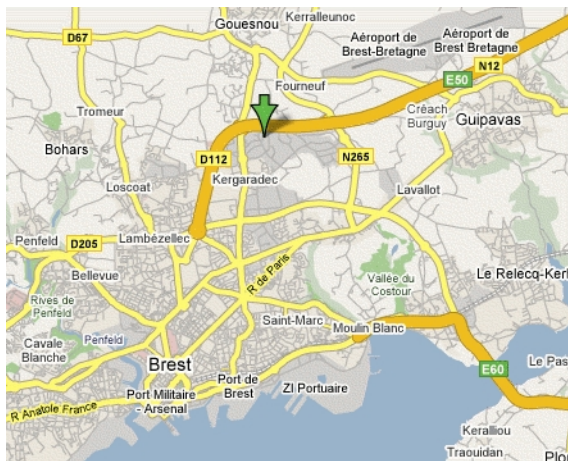
Pourquoi modéliser ?

- Mieux comprendre le système en développement
- Appréhender ces systèmes dans leur entièreté → modèles de systèmes complexes



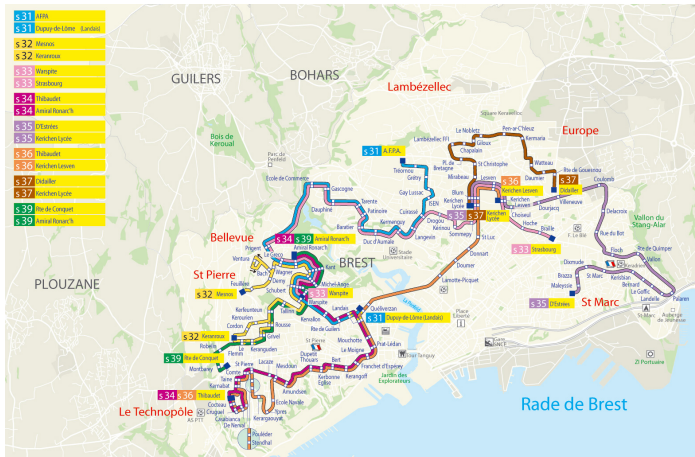
Pourquoi modéliser ?

- Mieux comprendre le système en développement
- Appréhender ces systèmes dans leur entièreté → modèles de systèmes complexes



Pourquoi modéliser ?

- Mieux comprendre le système en développement
- Appréhender ces systèmes dans leur entièreté → modèles de systèmes complexes



Des logiciels au génie logiciel

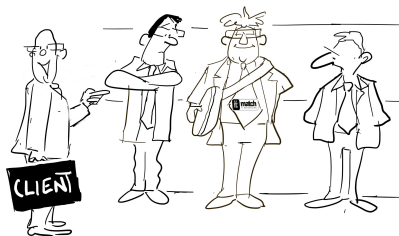


Etude projet informatique

- 16 % conformes
- 53 % dépassements
- 31 % abandon

⇒ Génie Logiciel

Des logiciels au génie logiciel

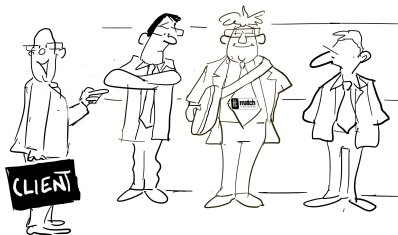


Etude projet informatique

- 16 % conformes
- 53 % dépassements
- 31 % abandon

⇒ Génie Logiciel

Des logiciels au génie logiciel

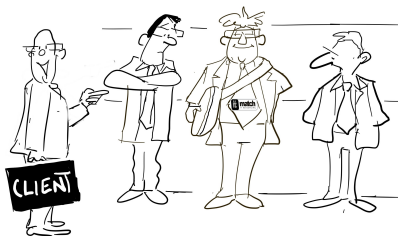


Etude projet informatique

- 16 % conformes
- 53 % dépassements
- 31 % abandon

⇒ Génie Logiciel

Des logiciels au génie logiciel



Etude projet informatique

- 16 % conformes
- 53 % dépassements
- 31 % abandon

⇒ Génie Logiciel

- 1 Introduction
- 2 Diagramme de classes (4 UC)
- 3 Diagramme de cas d'utilisation (1 UC)
- 4 Diagrammes d'interaction (2 UC)

- 1 Introduction
- 2 Diagramme de classes (4 UC)
- 3 Diagramme de cas d'utilisation (1 UC)
- 4 Diagrammes d'interaction (2 UC)

UML : Unified Modeling Language

- **Langage graphique** de modélisation pour
 - spécifier,
 - concevoir,
 - construire,
 - et documenter
 - des applications informatiques
- **Synthèse** des *bonnes pratiques* de l'ingénierie informatique
- **Unification** de modèles
- **Standardisation** par l'OMG (*Object Management Group*)

Objectifs

- Fournir un langage **visuel** et expressif
- Fournir des mécanismes d'**extension**
- Etre **indépendant** des technologies et langages d'implémentation
- Fournir une base **formelle** pour la modélisation

3 axes de modélisation

Approche classique - projet

① Fonctionnel

- Cahier des charges : diag. cas d'utilisation → scénarios écrits
- Scénarios formels : diag. seq / comm → objets/classes

② Statique

- classes : diag. classes

③ Dynamique

- dynamique chaque objet : diag. états/transitions
- dynamique globale : diag. activités

Pour quelles tâches ?

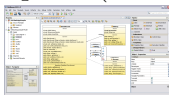
- Conception (« forward engineering »)



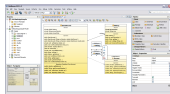
```
class classC {  
    private:  
    someOther someOther;  
    someOther someOther();  
    classC() {someOther();}  
    public:  
    classC();  
    classC() {someOther();}  
    someOther() {someOther();}  
};  
  
class classTrack : public classC {  
    someOther someOther;  
    public:  
    classTrack();  
    classTrack() {someOther();}  
};
```

- Rétro conception (« reverse engineering »)

```
class classC {  
    private:  
    someOther someOther;  
    someOther someOther();  
    classC() {someOther();}  
    public:  
    classC();  
    classC() {someOther();}  
    someOther() {someOther();}  
};  
  
class classTrack : public classC {  
    someOther someOther;  
    public:  
    classTrack();  
    classTrack() {someOther();}  
};
```



- Documentation d'un système



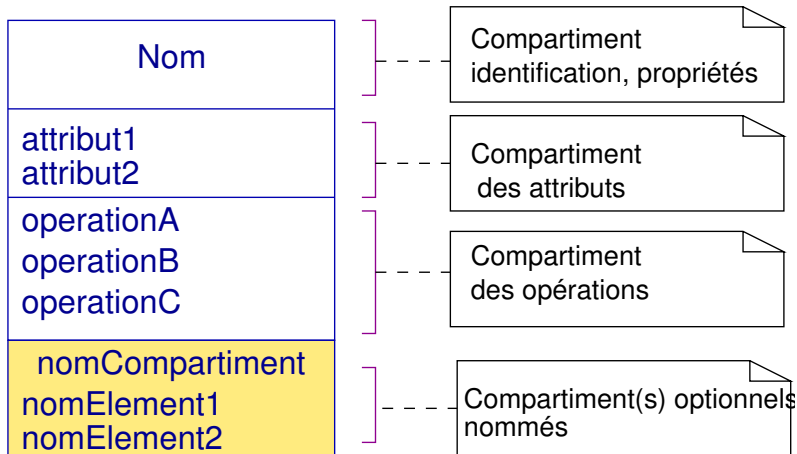
Plan

- 1 Introduction
- 2 Diagramme de classes (4 UC)
- 3 Diagramme de cas d'utilisation (1 UC)
- 4 Diagrammes d'interaction (2 UC)

Plan

- 1 Introduction
- 2 Diagramme de classes (4 UC)
- 3 Diagramme de cas d'utilisation (1 UC)
- 4 Diagrammes d'interaction (2 UC)

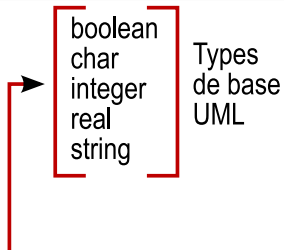
Classe, attribut et opération : notations



Attribut : syntaxe

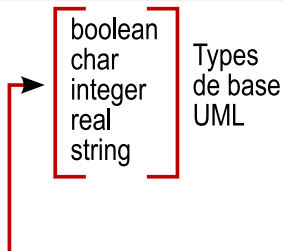
nom

Attribut : syntaxe

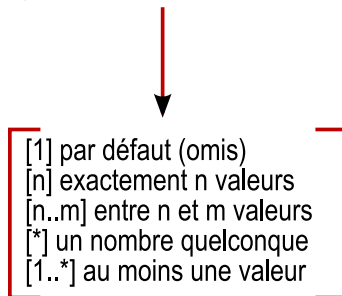


nom : type

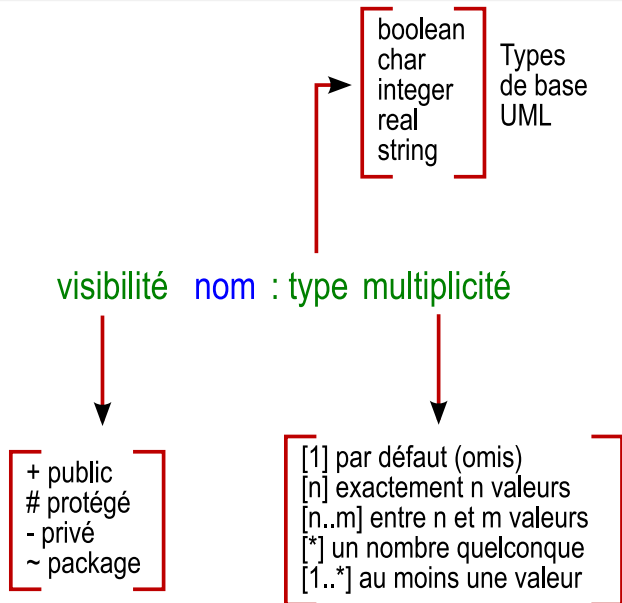
Attribut : syntaxe



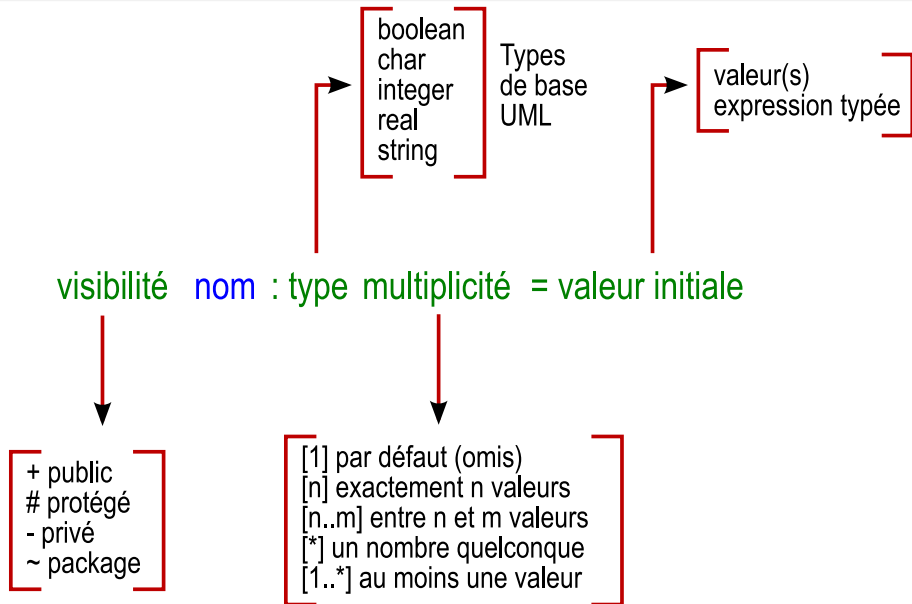
nom : type multiplicité



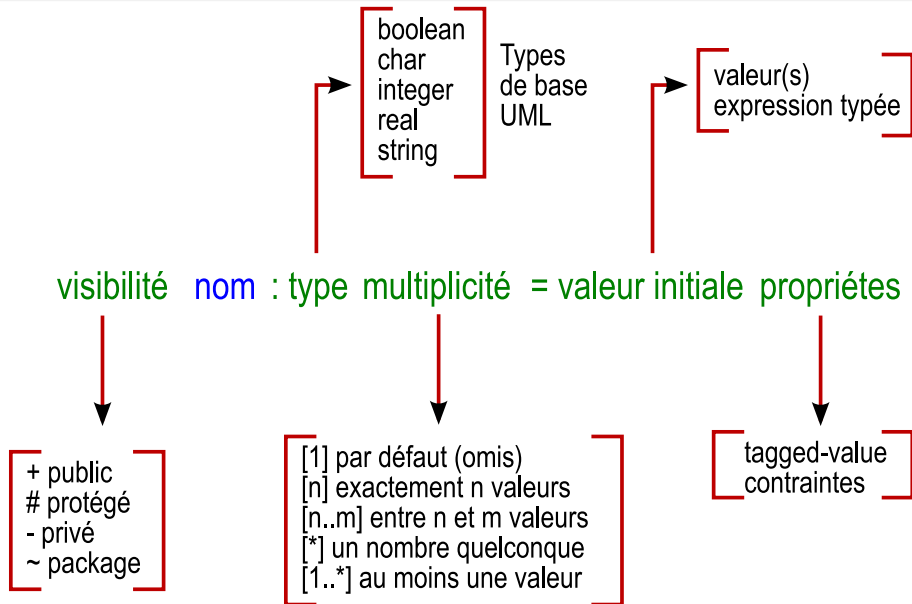
Attribut : syntaxe



Attribut : syntaxe



Attribut : syntaxe



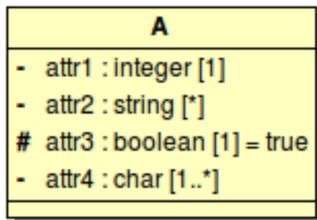
Attribut : visibilité

Accessibilité : quels éléments peuvent le référencer ?

+	public	tout élément qui accède à la classe
#	protected	seul un élément de la classe ou de ses descendants
-	private	seul un élément de la classe
~	package	seul un élément du même package que la classe

élément : une classe qui référence la classe considérée

Exemple

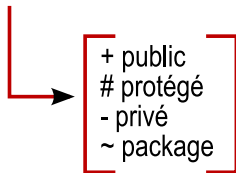


Opération : syntaxe

nom

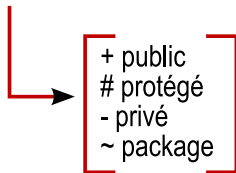
Opération : syntaxe

visibilité nom



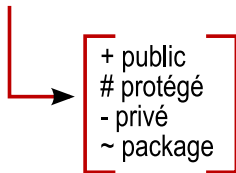
Opération : syntaxe

visibilité nom(par1, ..., parN)



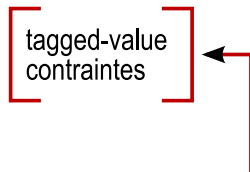
Opération : syntaxe

visibilité nom(par1, ..., parN) : type_ret [mult_ret]



Opération : syntaxe

tagged-value
contraintes



visibilité nom(par1, ..., parN) : type_ret [mult_ret] propriétés



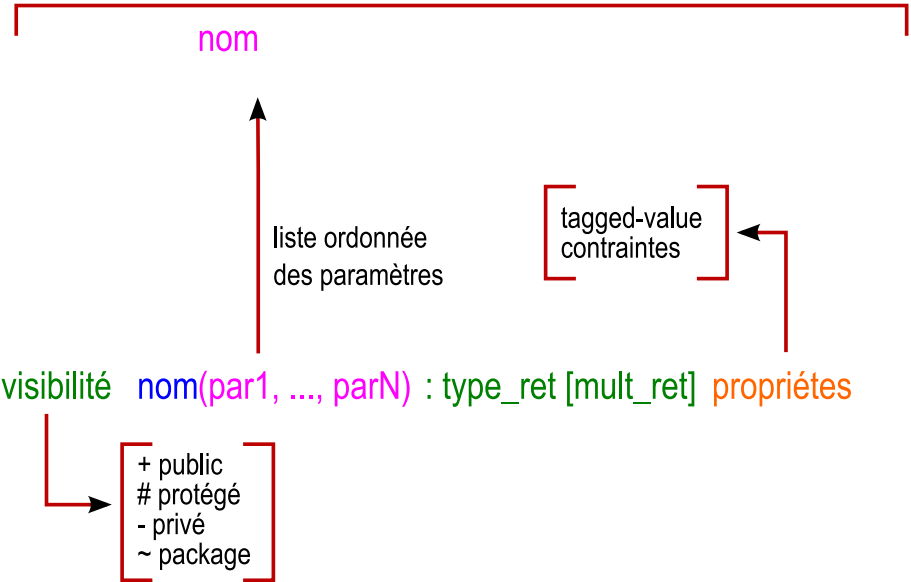
+ public
protégé
- privé
~ package

Opération : syntaxe

nom

liste ordonnée
des paramètrestagged-value
contraintes

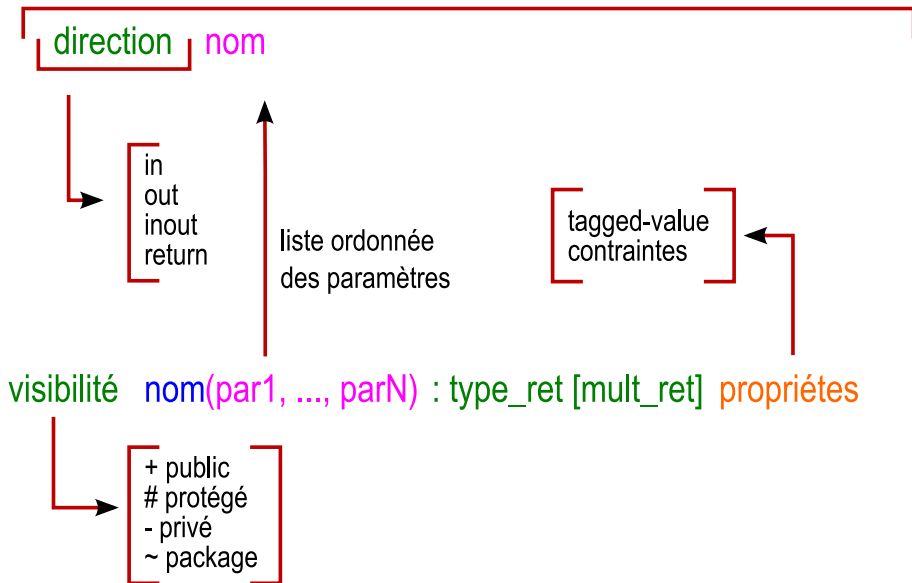
visibilité nom(par1, ..., parN) : type_ret [mult_ret] propriétés



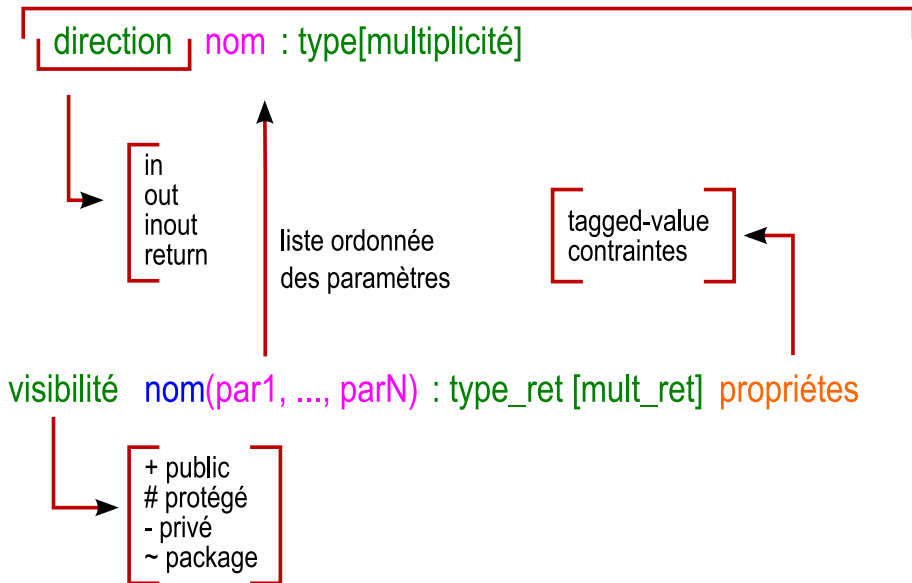
```

+ public
# protégé
- privé
~ package
  
```

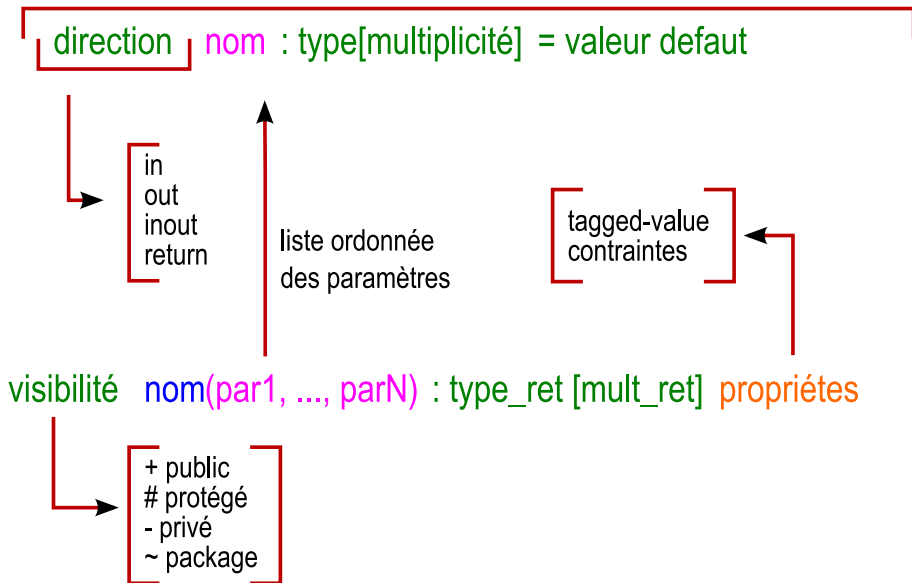
Opération : syntaxe



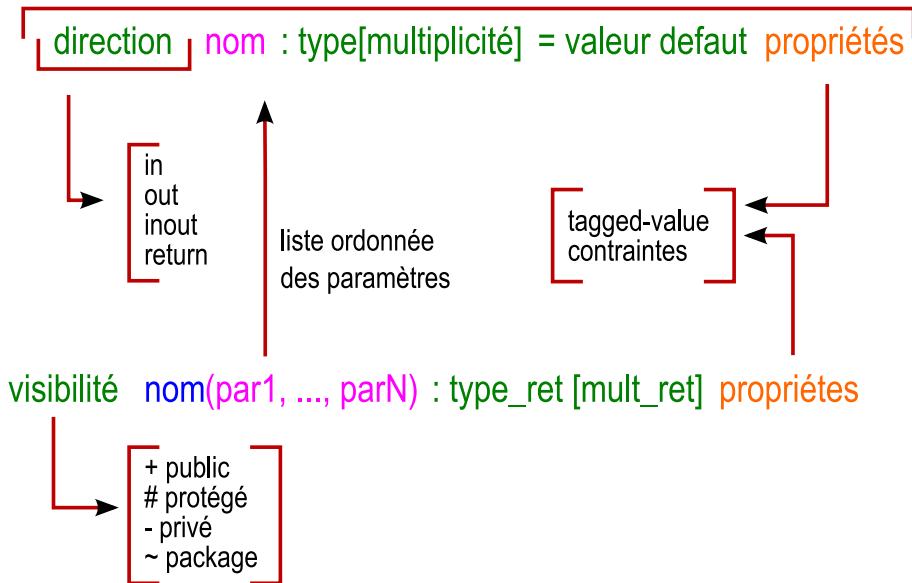
Opération : syntaxe



Opération : syntaxe



Opération : syntaxe



Opération : direction des paramètres

- information que l'objet serveur ne possède pas, mais qui est nécessaire à la réalisation de l'opération : **direction = in**
- information nécessaire à la réalisation de l'opération et transformée par celle-ci : **direction = inout**
- information produite par l'exécution de l'opération, donc inexistante avant ; **direction = out ou return**

Exemple

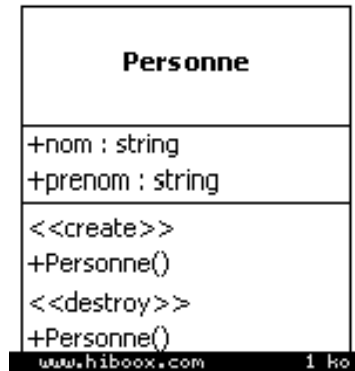
A

+ ope1()

+ ope2(in para1 : string[1])

ope3(in para1 : integer[1] = 0, inout para

Exemple



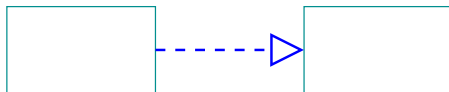
Types



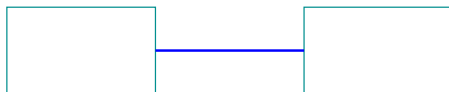
Dépendance



Généralisation

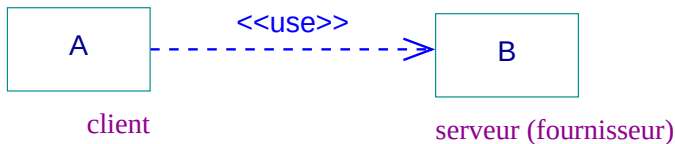


Réalisation



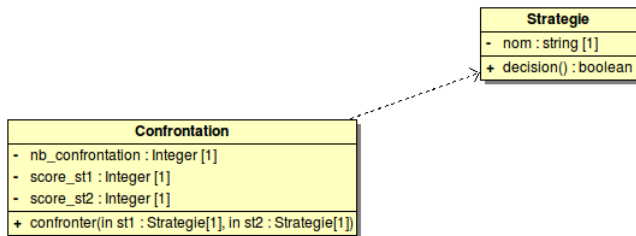
Association

Relation de dépendance entre classes (*classifier*)



- Indique une dépendance entre les propriétés d'une classe (le *client*) et une autre classe (le *serveur*, *supplier*)
- En conséquence, une modification du *serveur* peut affecter le comportement du *client*
- Exemples :
 - une opération de la classe A fait appel à une opération de la classe B
 - une opération de A a comme paramètre un objet B

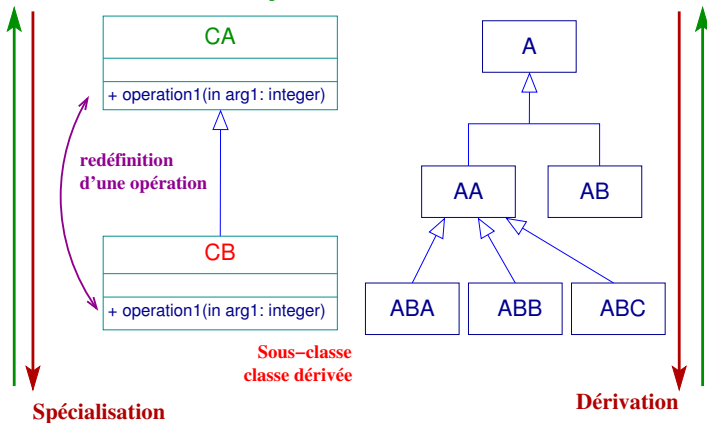
Exemple de dépendance entre classes



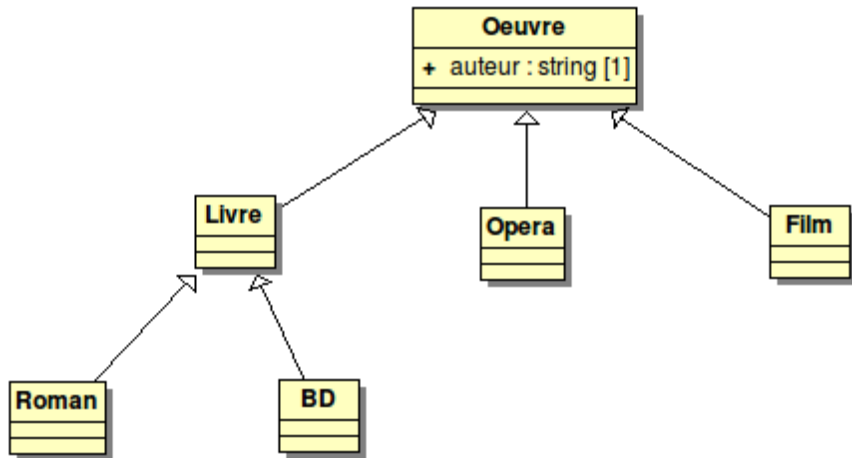
Séréotypes de dépendance entre classes

<code>access</code>	import du contenu d'un autre package
<code>create</code>	la classe crée des instances d'une autre classe
<code>instantiate</code>	la méthode d'une classe crée des instances d'une autre
<code>permit</code>	donne accès aux éléments privés
<code>use</code>	un élément requiert un autre élément

Généralisation – spécialisation

Généralisation**Super-classe****Héritage**

Généralisation – spécialisation : exemple



Généralisation : vocabulaire



A	est une spécialisation	de B
---	------------------------	------

A	est une sous-classe	de B
---	---------------------	------

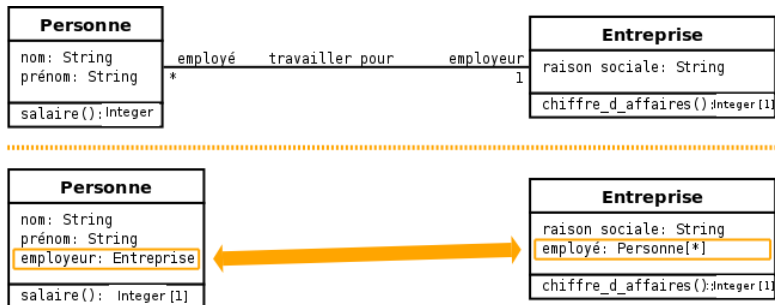
A	dérive	de B
---	--------	------

A	hérite	de B
---	--------	------

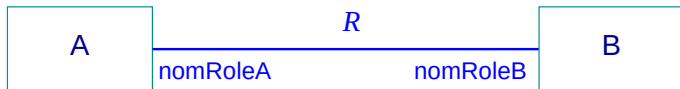
B	est une généralisation	de A
---	------------------------	------

B	est une super-classe	de A
---	----------------------	------

Association : principe



Association : rôles



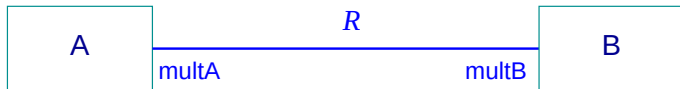
nomRole : indique ce que représente l'ensemble des instances associées à une instance de la classe par la relation *R*.

L'accès peut être +, # ou -

nomRoleB nom de l'ensemble des instances de la classe B qui sont en relation avec 1 instance de la classe A par la relation *R*.

nomRoleA nom de l'ensemble des instances de la classe A qui sont en relation avec 1 instance de la classe B par la relation *R*.

Association : multiplicités



valeurs possibles du cardinal de l'ensemble des instances associées à une instance de la classe par la relation R .

multB cardinal de l'ensemble des instances de la classe B qui sont en relation avec 1 instance de la classe A par la relation R .

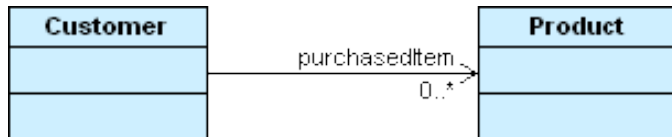
multA cardinal de l'ensemble des instances de la classe A qui sont en relation avec 1 instance de la classe B par la relation R .

Multiplicité : notation

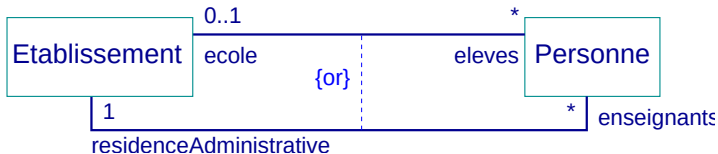
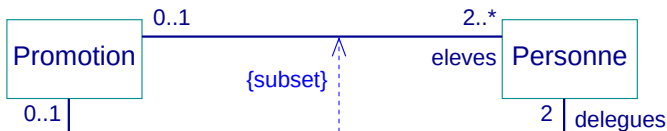
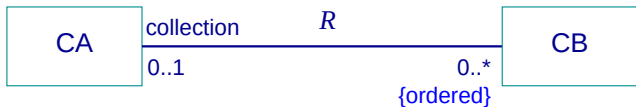
notation : min .. max

	abreg.	signification
1..1	1	exactement 1
0..1	–	zéro ou un (optionnel)
0..*	*	aucun ou plusieurs
1..*	–	au moins 1
n..m, p	–	entre n et m ou exactement p

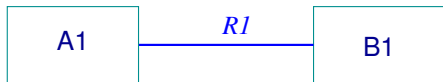
Association unidirectionnelle



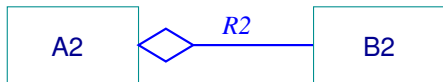
Contraintes sur une association



Types d'association



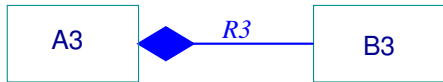
Association



Agrégation

Agrégat

élément



Composition

Composite

Composant

Agrégation

Agrégation : association simple

- + contraintes d'intégrité
- + graphe acyclique
- + éléments partageables

Composition

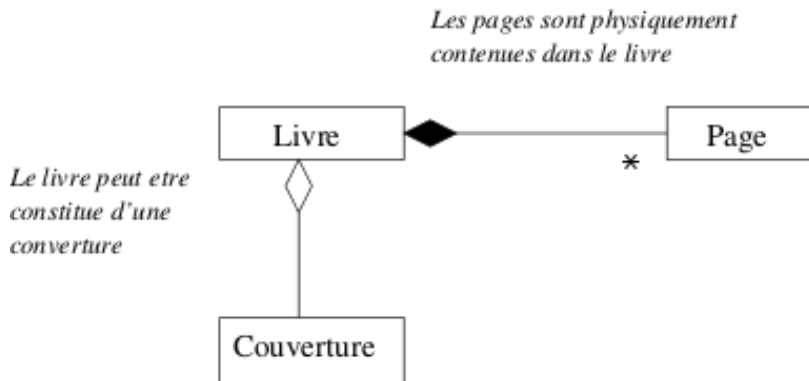
Composition : agrégation

- + contrainte de durée de vie
- + composants non partageables

Agregation/composition : exemple

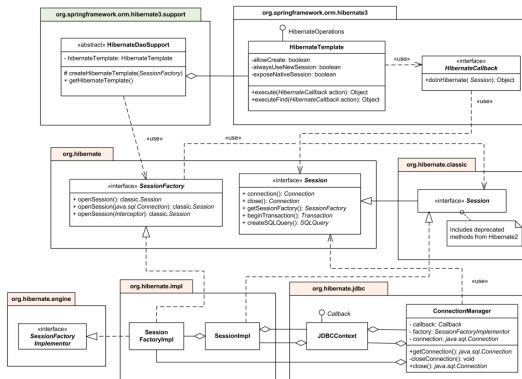


Agregation/composition : exemple

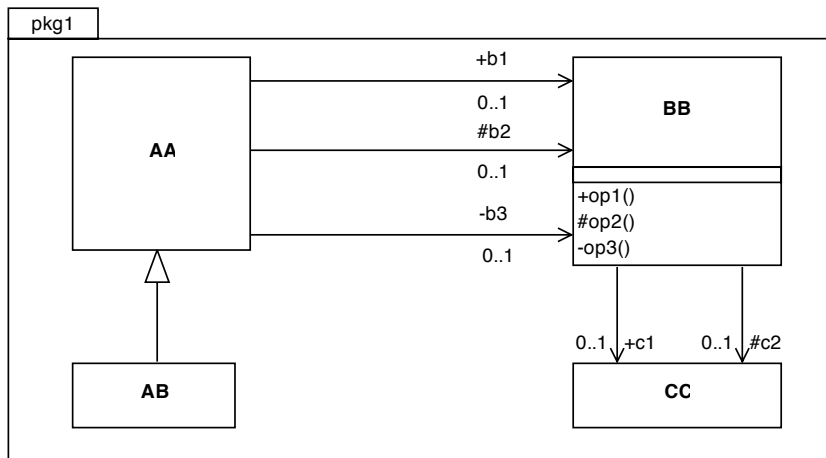


Package

- grouper dans des ensembles cohérents.
- structurer les diagrammes et donnent une vision globale plus claire.



Navigation



Classe abstraite

Propriété optionnelle d'une classe

- Définition :

- classe non instanciable
- ensemble de propriétés communes à différentes classes mais partiellement définies.
- donc, seuls des objets d'une classe dérivée sont instanciables

- Deux raisons :

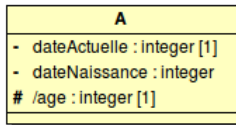
- ① bien que l'instanciation d'un tel objet serait possible cela n'aurait pas de sens
ex. *Personne* – *Eleve* – *Prof*
- ② au moins une des propriétés de la classe n'est pas définie
ex. *Shape* opération *draw*

- Notation :

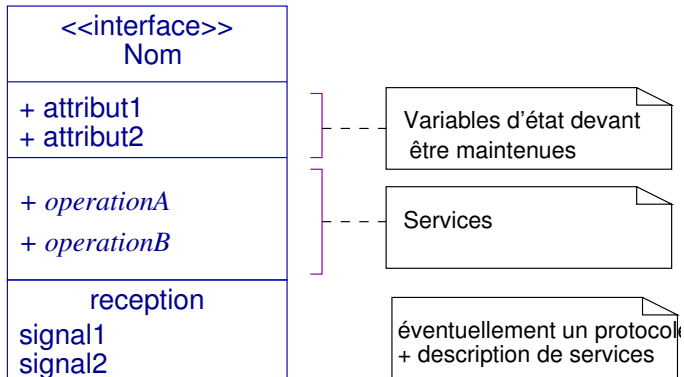
- `<< abstract >>` : tagged-value placée après le nom de la classe
- *NomClass* : en italique

Attribut dérivé

- sa valeur se calcule à partir d'autres propriétés de la classe (attributs ou autres)
- symbolisés par l'ajout d'un $\ll / \gg$ devant leur nom



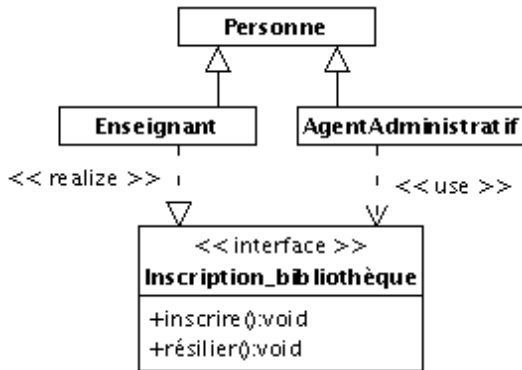
Interface



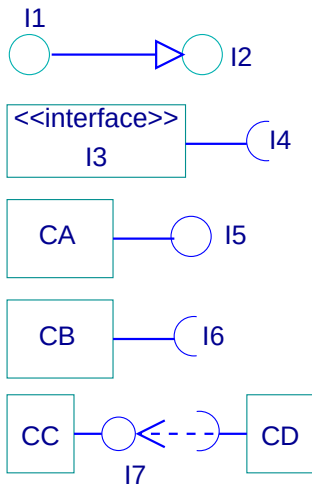
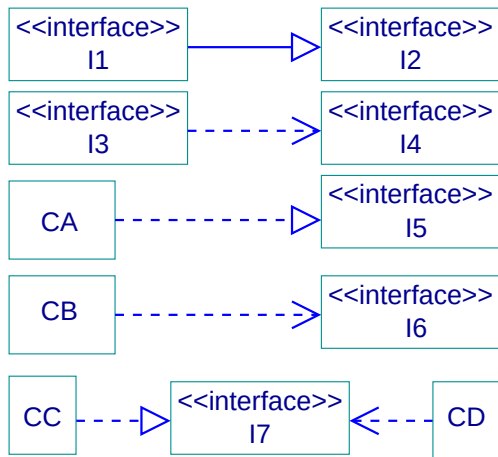
Relation de réalisation : entre classes et interfaces



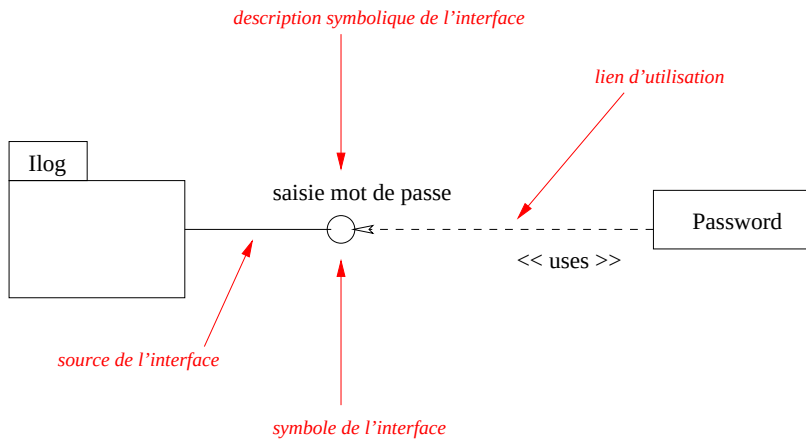
Interface : Exemple



Relations avec une interface

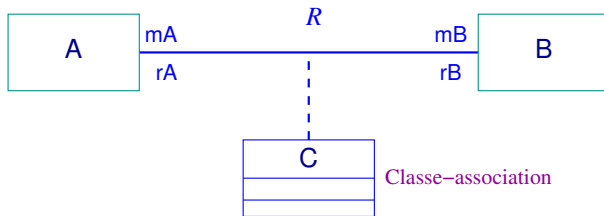


Interface : exemple



Classe-association

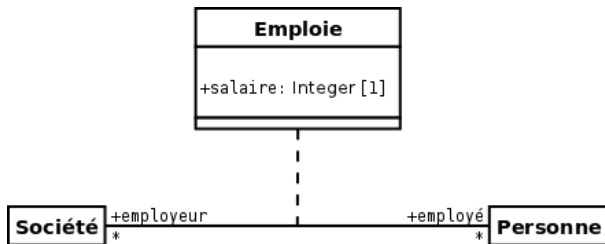
cas particuliers d'association



- L'association entre les classes A et B est réalisée par **un** objet de la classe C (sous sa responsabilité)
- la classe C a des propriétés qui lui sont propres (attributs, opérations...)

Classe-association

exemple

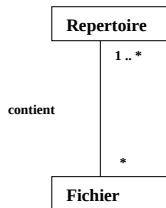


Association qualifiée

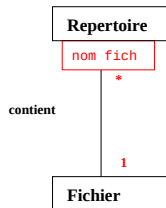


- **Clé** : attribut de la relation R permettant de définir (*qualifier*) le sous-ensemble des instances de la classe B (rôle rB) associées à 1 instance de la classe A
- le couple (Instance de A , valeur Clé) identifie le sous-ensemble rB .
- **Multiplicité** : $mB = \text{cardinal de } rB$
- **Exemple** : annuaire inversé

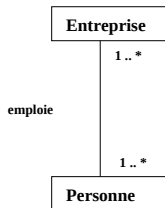
Association qualifiée : exemple



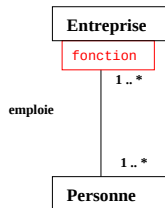
sans qualifiant



avec qualifiant



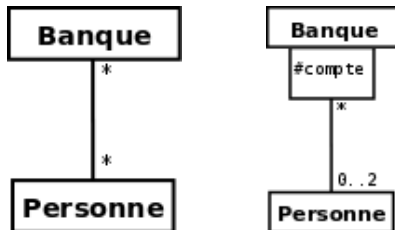
sans qualifiant



avec qualifiant

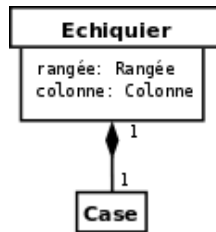
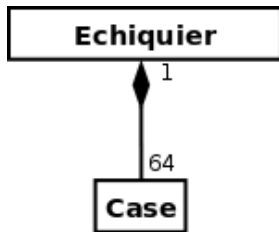
Association qualifiée

exemple



Association qualifiée

exemple



Démarche pour bâtir un diagramme de classes (1/2)

① Trouver les classes du domaine étudié

- En collaboration avec un expert du domaine.
- Les classes correspondent généralement à des *concepts* ou des *substantifs* du domaine.

② Trouver les associations entre classes

- Les associations correspondent souvent à des verbes, ou des constructions verbales, mettant en relation plusieurs classes, comme « est composé de », « pilote », « travaille pour ».
- Attention, méfiez vous de certains attributs qui sont en réalité des relations entre classes.

Démarche pour bâtir une diagramme de classes (2/2)

③ Trouver les attributs des classes

- Les attributs correspondent souvent à des substantifs, ou des groupes nominaux, tels que « la masse d'une voiture » ou « le montant d'une transaction ».
- Les adjectifs et les valeurs correspondent souvent à des valeurs d'attributs.

④ Organiser et simplifier le modèle

- En éliminant les classes redondantes et en utilisant l'héritage.

⑤ Vérifier les chemins d'accès aux classes

⑥ Itérer et raffiner le modèle

- Un modèle est rarement correct dès sa première construction.

Plan

- 1 Introduction
- 2 Diagramme de classes (4 UC)
- 3 Diagramme de cas d'utilisation (1 UC)
- 4 Diagrammes d'interaction (2 UC)

Plan

- 1 Introduction
- 2 Diagramme de classes (4 UC)
- 3 Diagramme de cas d'utilisation (1 UC)
- 4 Diagrammes d'interaction (2 UC)

Exprimer les besoins

- Comment donner un moyen simple d'exprimer les besoins d'utilisateurs non informaticiens ?
- Première étape UML d'analyse d'un système

Exigences fonctionnelles

Modèle construit en phase de définition des exigences fonctionnelles et enrichi pendant la phase d'analyse en utilisant d'autres modèles (entre-autres les modèles d'interaction)

Objectifs :

- ❶ identifier les fonctionnalités du logiciel
- ❷ en définir le périmètre
- ❸ identifier les éléments externes en interaction directe

Acteur

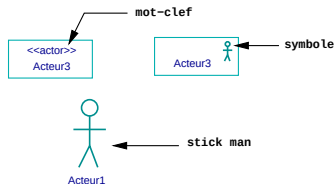
● Définition :

- rôle joué par une entité externe qui interagit avec le système
- il peut consulter et/ou modifier l'état du système par messages

● Comment les identifier ?

- utilisateurs humains
- systèmes connexes qui interagissent également avec le système

● Comment les représenter ?



Exemple : Client, Conseiller financier, SI Banque ...

Cas d'utilisation

- **Définition :**

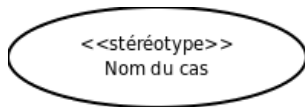
- Séquences d'actions réalisées par le système (résultat observable pour un acteur)
- Comportement attendu ($\neq$ mode de réalisation) :
 - *ce que* le futur devra faire
 - *pas comment* il le fera

Cas d'utilisation

• Comment les identifier ?

- Ensemble des cas utilisation == exigences fonctionnelles du système
- Un cas == fonction métier selon le point de vue des acteurs
- Pour chaque acteur :
 - rechercher ses utilisations métiers
 - déterminer dans le cahier des charges les services attendus
- Nommez les cas d'utilisation (point de vue acteur) :
verbe à l'infinitif + complément

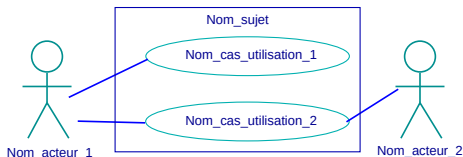
• Comment les représenter ?



Exemple : Consulter un compte, Retirer de l'argent, Déposer un chèque

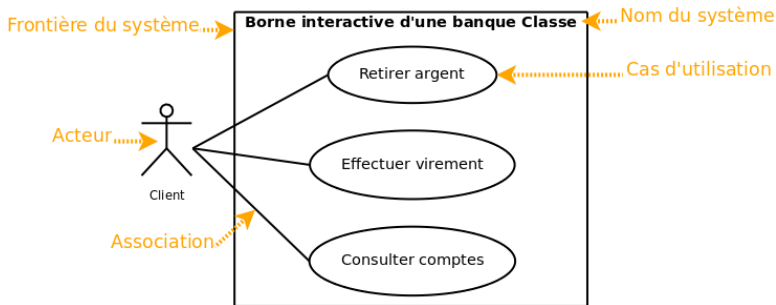
...

Cas d'utilisation et acteurs



- **Acteur :**
élément **externe** en **interaction directe** avec le sujet
- **Cas d'utilisation :**
ensemble **fonctionnel** cohérent, identifiable extérieurement
et fourni par un **classeur** (le *sujet*)
- **Association Acteur – Cas :**
chemin de communication indiquant la participation de l'acteur
à la réalisation du cas

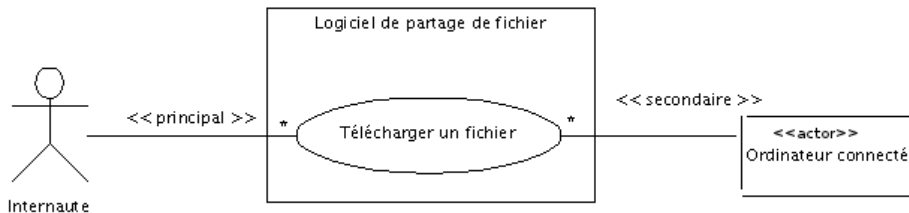
Association : Exemple



Acteurs principaux et secondaires

- Un acteur est qualifié de **principal** pour un cas d'utilisation lorsque ce cas rend service à cet acteur.
- Les autres acteurs sont alors qualifiés de **secondaires**.
- Un cas d'utilisation a au plus un acteur principal.
- Le stéréotype « primary » vient orner l'association reliant un cas d'utilisation à son acteur principal
- Le stéréotype « secondary » est utilisé pour les acteurs secondaires

Acteurs principaux et secondaires : Exemple



Types et représentation



- CA1 *inclut* CB1
1 réalisation de CA1
⇒ 1 réalisation de CB1



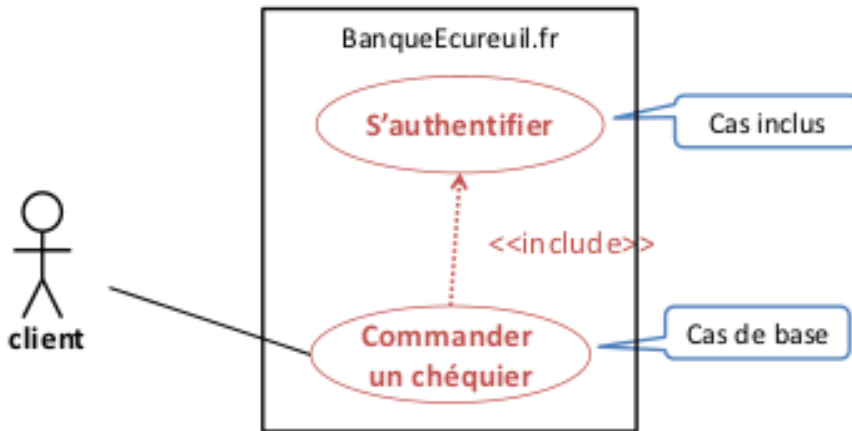
- CA2 *étend* CB2
dans un certain contexte
Selon le contexte,
on réalise soit CA2, soit CB2



- CA3 *spécialise* CB3
Selon le contexte,
on réalise soit CA3, soit CB3

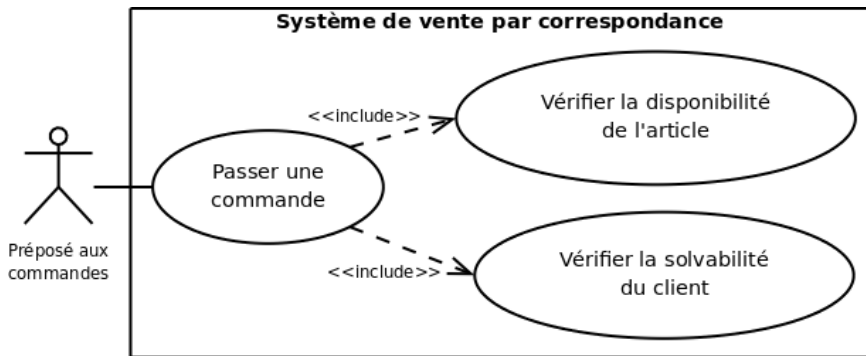
Exemple relation d'inclusion

Le cas inclus est ajouté obligatoirement au cas de base



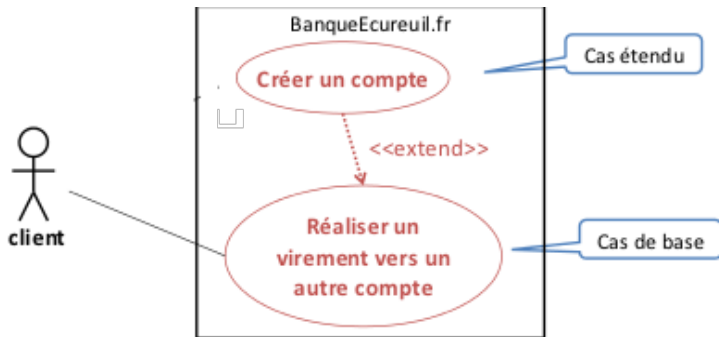
Exemple relation d'inclusion

Identifier une partie commune aux différents cas d'utilisation et de la factoriser dans un nouveau cas inclus dans ces derniers.



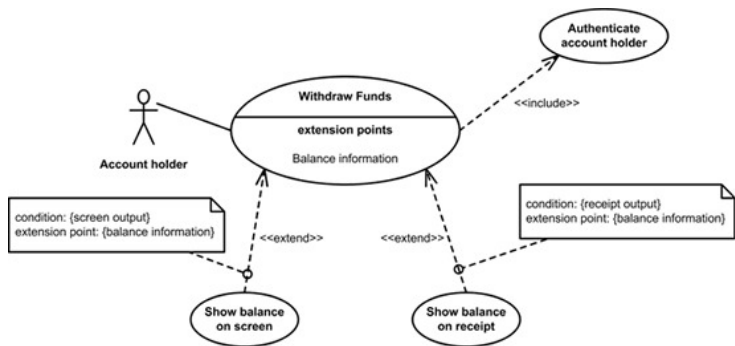
Exemple relation d'extension

Enrichir un cas d'utilisation par un autre, cependant, cet enrichissement est optionnel.



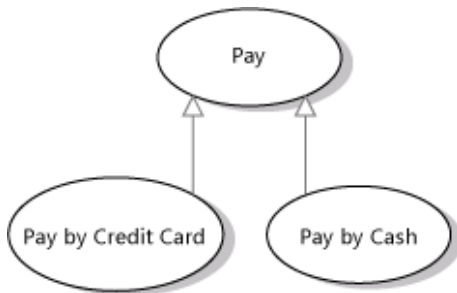
Exemple relation d'extension

L'extension se fait dans le cas d'utilisation de base, en un point précis appelé point d'extension

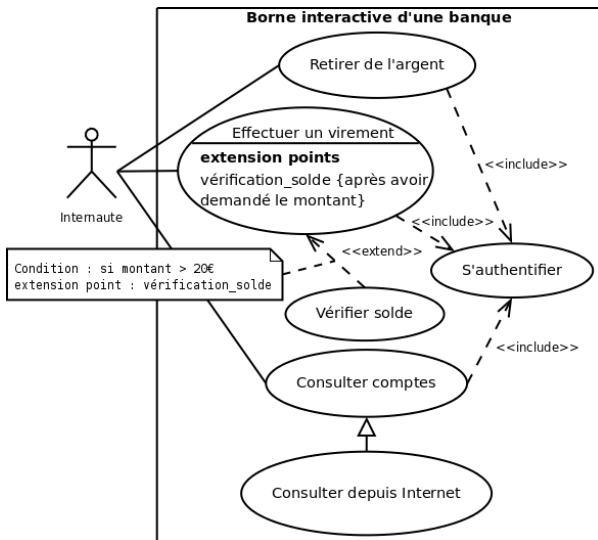


Exemple relation Généralisation/Spécialisation

Formaliser les variations importantes sur le même cas d'utilisation



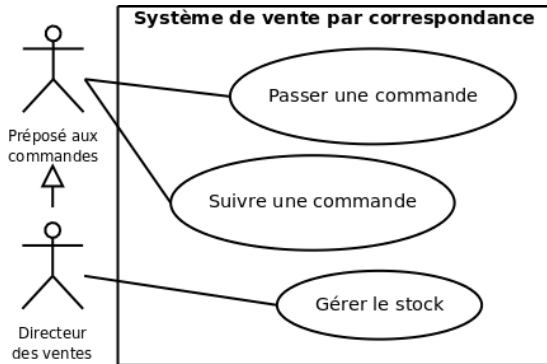
Exemple complet



Généralisation

- La seule relation possible entre deux acteurs est la généralisation
 - un acteur A est une généralisation d'un acteur B si l'acteur A peut être substitué par l'acteur B
 - tous les cas d'utilisation accessibles à A le sont aussi à B, mais l'inverse n'est pas vrai.

Généralisation : Exemple



Scénario



- **Définition :**

succession particulière d'enchaînements s'exécutant du début à la fin du cas

- **Un cas d'utilisation contient :**

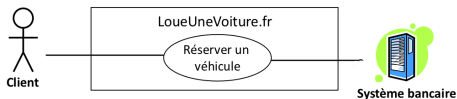
- un scénario nominal
- plusieurs scénarios alternatifs (qui se terminent normalement)
- plusieurs scénarios d'erreur (qui se terminent par un échec)

En pratique ...

- La fiche de description textuelle d'un cas d'utilisation n'est pas normalisée...
- Cependant, on peut utiliser la structuration suivante :

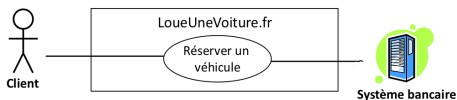
Sommaire d'identification (obligatoire)	Inclut titre, résumé, dates de création et de modification, version, responsable, acteurs ...
Description des scénarios (obligatoire)	Décrit le scénario nominal, les scénarios d'erreur, les pré/post-conditions
Exigences fonctionnelles non-fonctionnelles (optionnel)	Ajoute, si c'est pertinent, les informations suivantes : fréquence; disponibilité, fiabilité, confidentialité ...

Exemple scénario



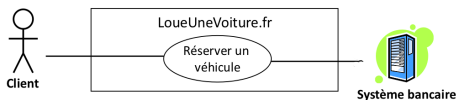
Sommaire	Scénario nominal « Réserver un véhicule »
Description	1. Le client saisit son code et son login d'identification 2. Le système vérifie le code et le login d'identification 3. Le système demande au client de saisir les informations sur la réservation 4. Le client saisit les informations sur la réservation 5. Le système interroge l'acteur système bancaire pour vérifier l'acompte 6. Le système bancaire donne une réponse favorable 7. Le système envoie au client, un message de confirmation de la demande

Exemple scénario



Sommaire	Scénario alternatif « Réserver un véhicule »
Description	SA1 : code d'identification erroné pour la première ou la deuxième fois SA1 démarre au point 2 du scénario nominal 3. Le système indique au client que le code est erroné, pour la première ou la deuxième fois. Le scénario nominal reprend au point 1.

Exemple scénario



Sommaire	Scénario d'erreur « Réserver un véhicule »
Description	SE1 : code d'identification erroné pour la troisième fois SE1 démarre au point 2 du scénario nominal 3. Le système indique au client que le code est erroné pour la troisième fois. Le cas d'utilisation se termine en échec (l'objectif n'est pas atteint).

Quand utiliser les cas d'utilisation ?

- En phase d'élaboration
- En discutant avec les utilisateurs

Remarque : Un projet de 10 années-hommes devrait comprendre environ 12 cas d'utilisation¹

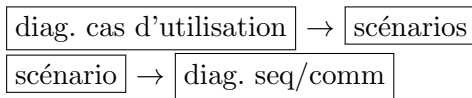
1. Résultat issu d'une commission de l'OOPSLA

Plan

- 1 Introduction
- 2 Diagramme de classes (4 UC)
- 3 Diagramme de cas d'utilisation (1 UC)
- 4 Diagrammes d'interaction (2 UC)

Plan

Du diag. de cas d'utilisation au diag. d'interactions



Cas d'utilisation *Piloter*

- scénario nominal pourrait être : un pilote démarre une voiture ce qui allume un moteur.
- Comment formaliser les communications entre instances (démarrer, allumer) ? → diag. de communication.
- Comment formaliser le séquençement des interactions (1 : démarrer ; 2 : allumer) ? → diag. séquence.
- Avant cela, il faut représenter les instances (objets) (un pilote, une voiture, un moteur)

Objet : instance de classifier

nom d'instance : nom de classe

Exemple

jean : Personne

pierre : Personne

Objets : instances de classier

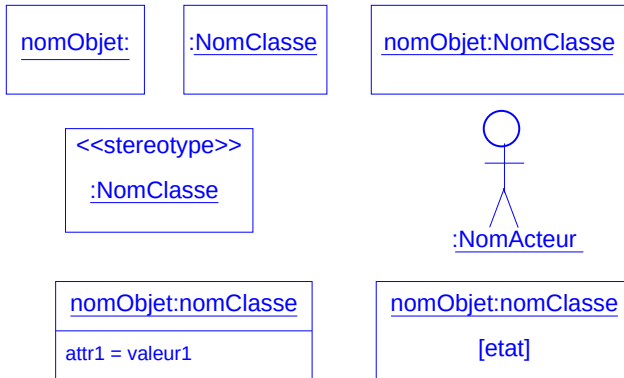
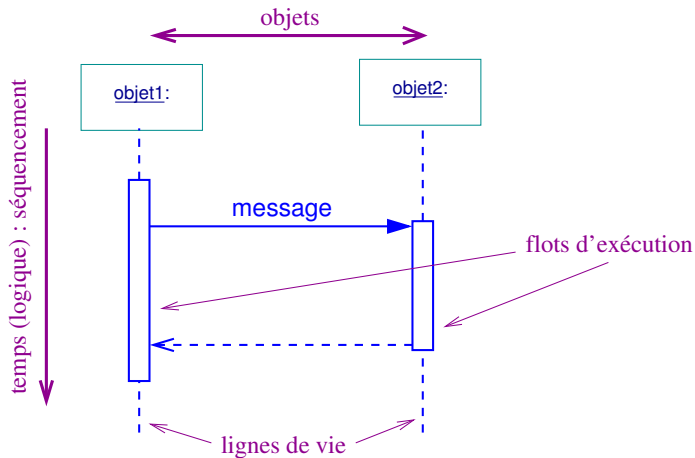


Diagramme de séquence : durée de vie et flots



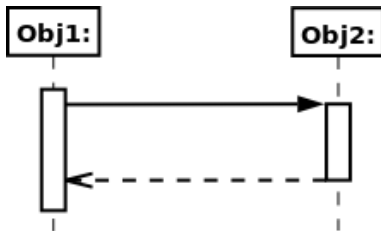
Messages

- l'envoi d'un signal ;
- l'invocation d'une opération ;
- la création ou la destruction d'une instance.

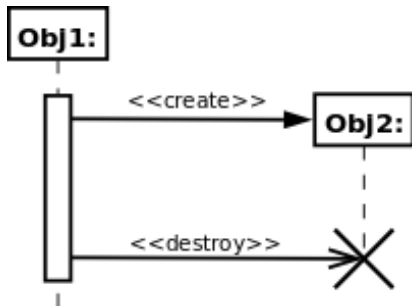
Messages asynchrones



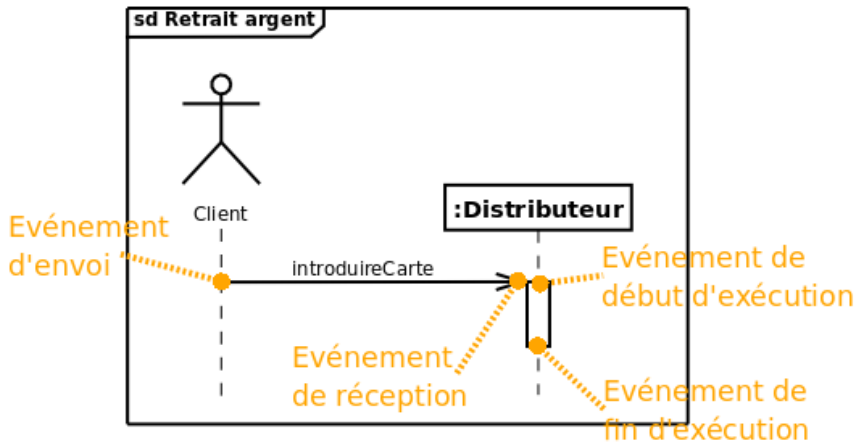
Messages synchrones



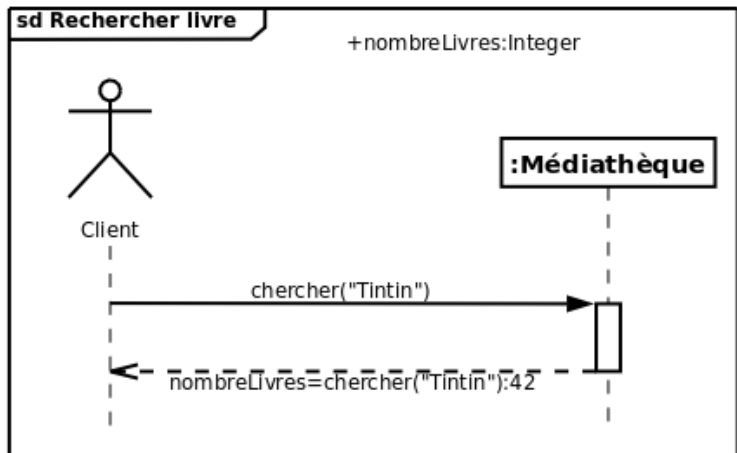
Messages de création et destruction d'instance



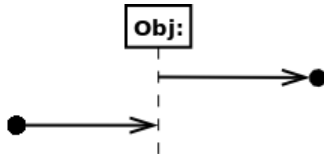
Événements et messages



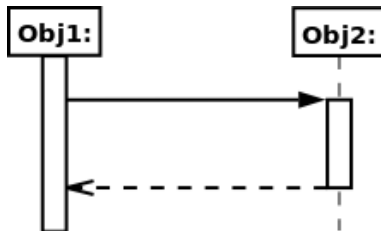
Syntaxe des messages et des réponses



Syntaxe des messages et des réponses



Exécution de méthode et objet actif



Exécution de méthode et objet actif

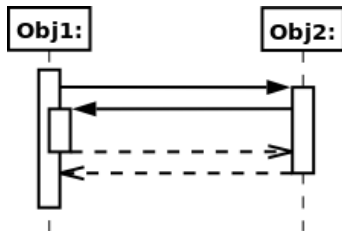


Diagramme de séquence : fragments d'interaction combinés

Articulation d'interactions par des **opérateurs** :

- choix et boucle : **alternative**, **option**, **break**, **loop**
- contrôle d'envoi en parallèle de messages : **parallel**, **critical region**
- contrôle d'envoi de messages : **ignore**, **consider**, **assertion**, **negative**
- ordre d'envoi des messages : **weak** sequencing, **strict** sequencing

Diagramme de séquence : structures de contrôle

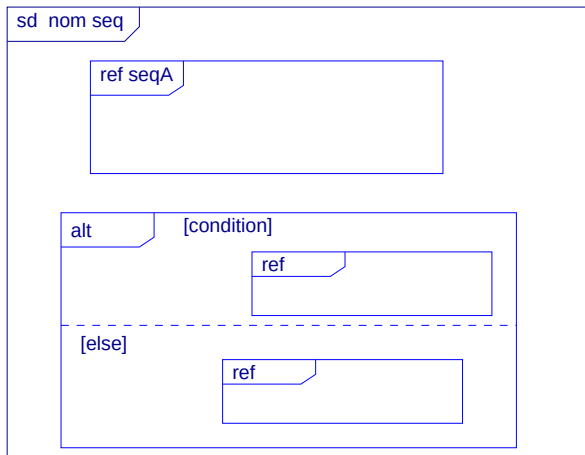


Diagramme de séquence : structures de contrôle (exemple)

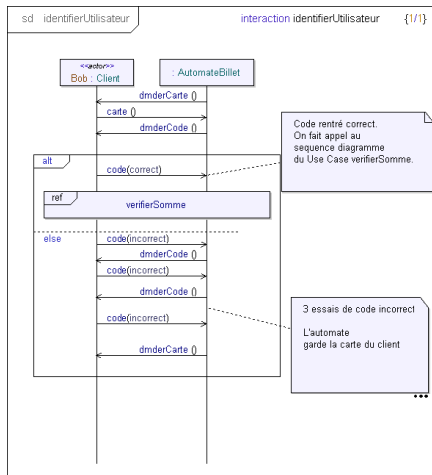


Diagramme de séquence : structures de contrôle (exemple)

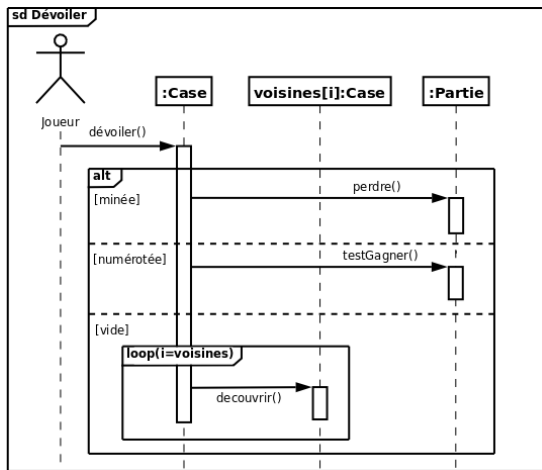


Diagramme de séquence : structures de contrôle

(exemple)

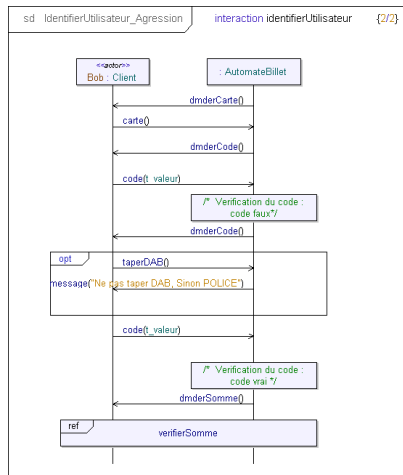


Diagramme de séquence : structures de contrôle (exemple)

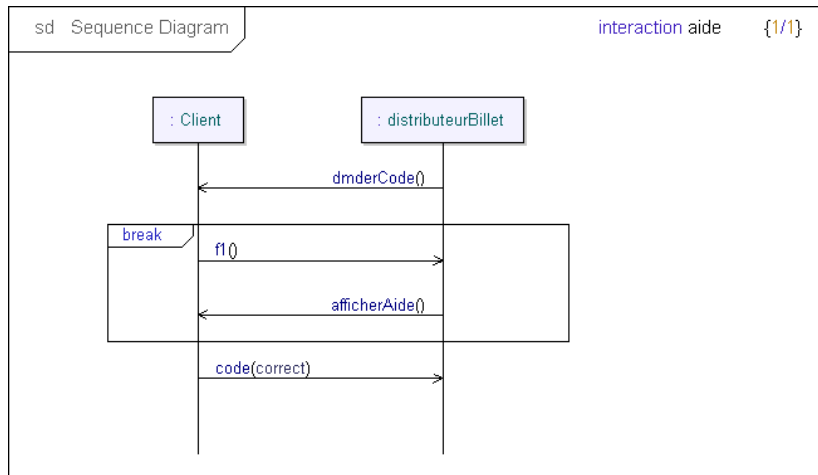


Diagramme de séquence : structures de contrôle (exemple)

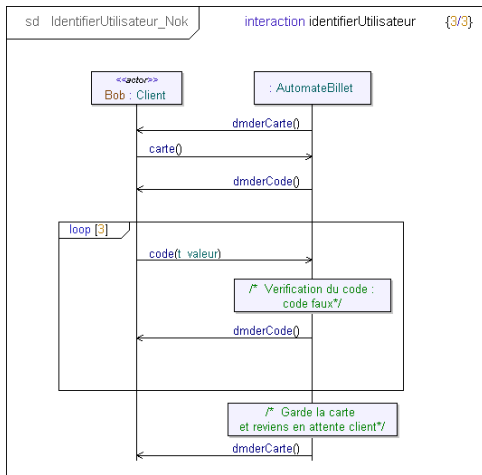


Diagramme de séquence : parallélisme et séquencement

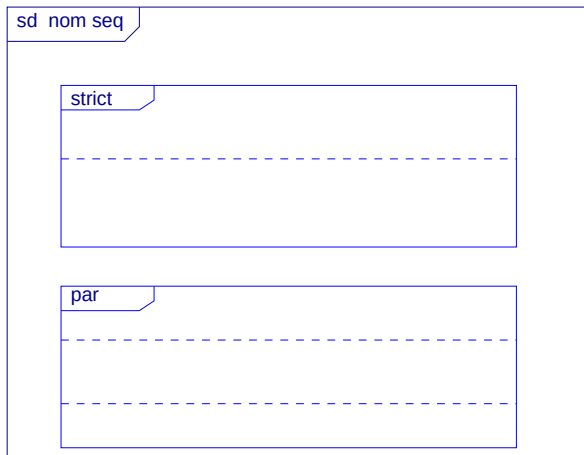


Diagramme de séquence : parallélisme et séquencement

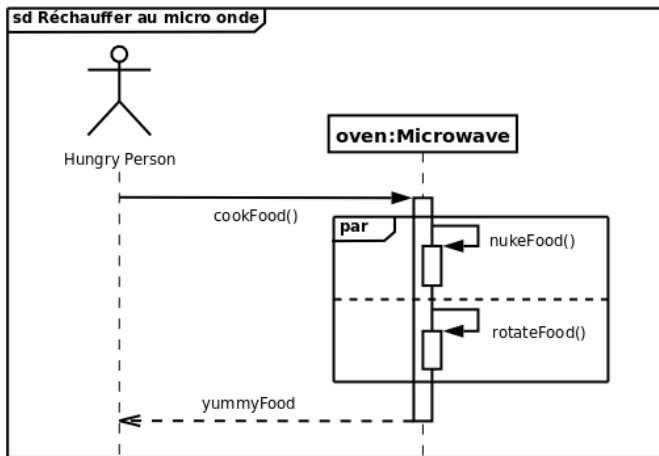


Diagramme de séquence : parallélisme et séquencement (exemple)

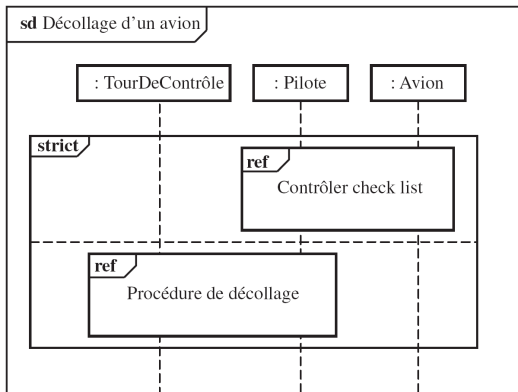


Diagramme de séquence : parallélisme et séquencement (exemple)

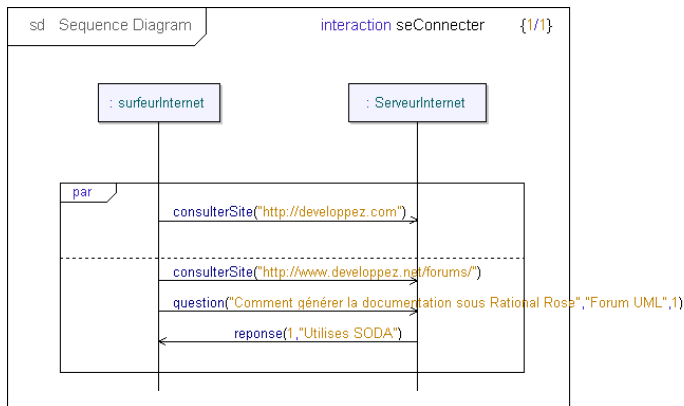


Diagramme de séquence : section critique (exemple)

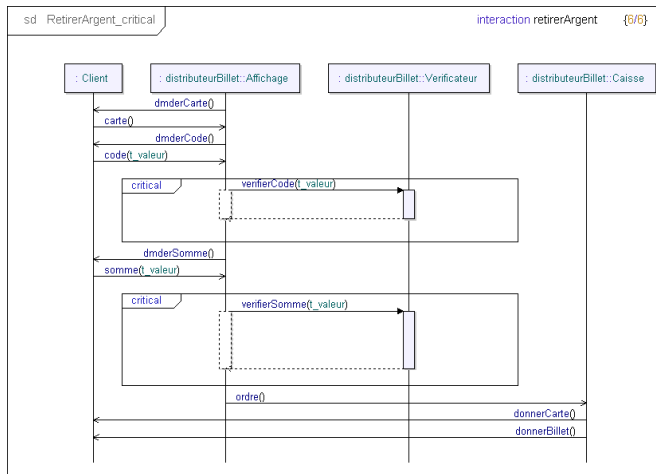


Diagramme de séquence : Ignorer (exemple)

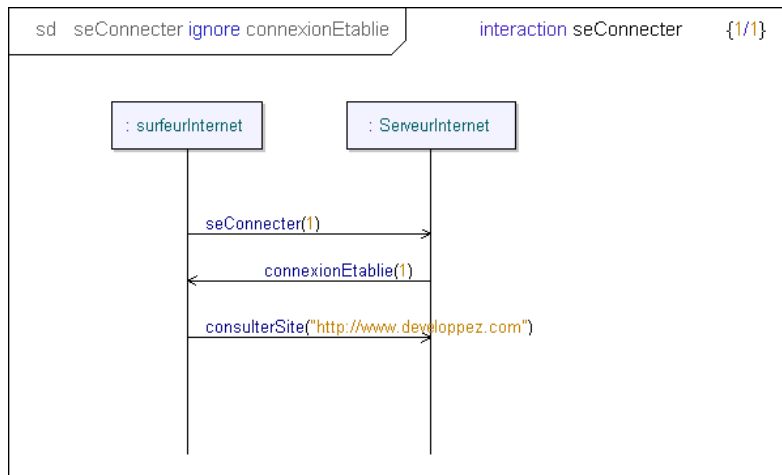


Diagramme de séquence : Considérer (exemple)

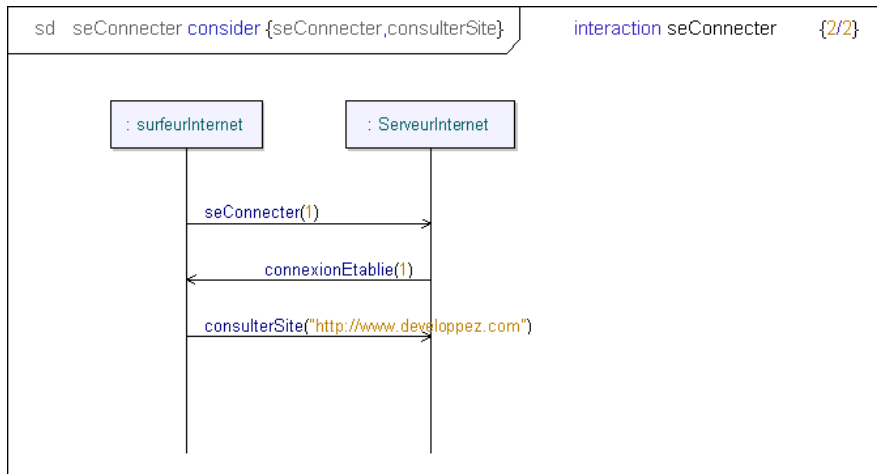


Diagramme de séquence : Négatif (exemple)

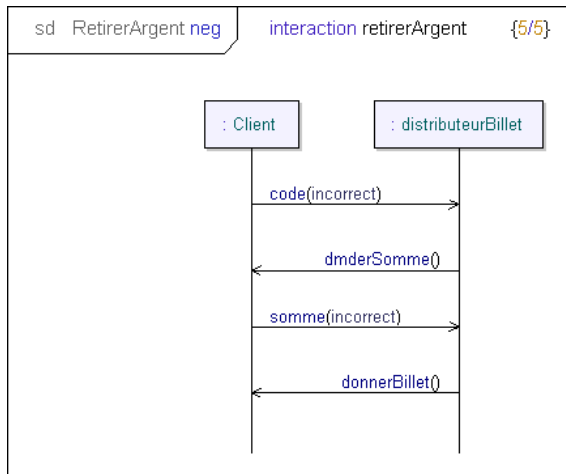


Diagramme de séquence : invariants d'état

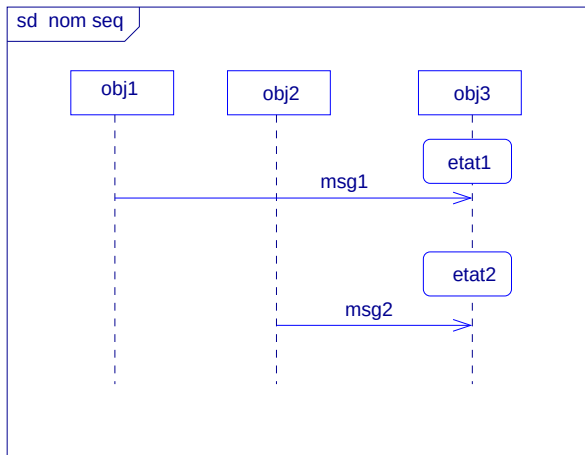


Diagramme de communication : Lignes de vie

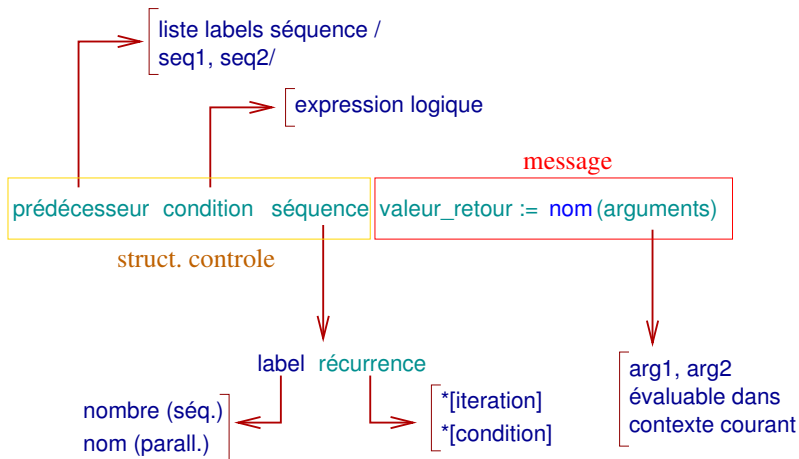
- Les lignes de vie sont représentées par des rectangles contenant une étiquette dont la syntaxe est :

[<nom_du_rôle>] : [<Nom_du_type>]

Diagramme de communication : Connecteurs

- Les relations entre les lignes de vie sont appelées **connecteurs**
 - se représente par un trait plein reliant deux lignes de vies
 - les extrémités peuvent être ornées de multiplicités.

Communication : interactions entre objets par messages



Messages : structures de contrôle — exemples

condition		récur. message
$[x > 0]$		$: \text{msg}()$
	*	$: \text{msg}()$
	*	$[x > 0] : \text{msg}()$
$[x > 0]$	*	$[i := 1..n] : \text{msg}()$

Diagramme de communication : exemple

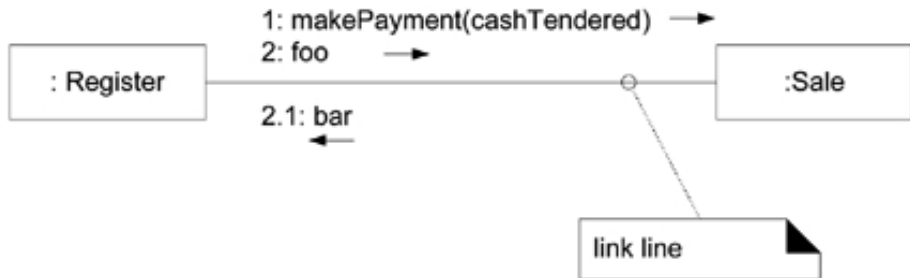


Diagramme de communication : exemple

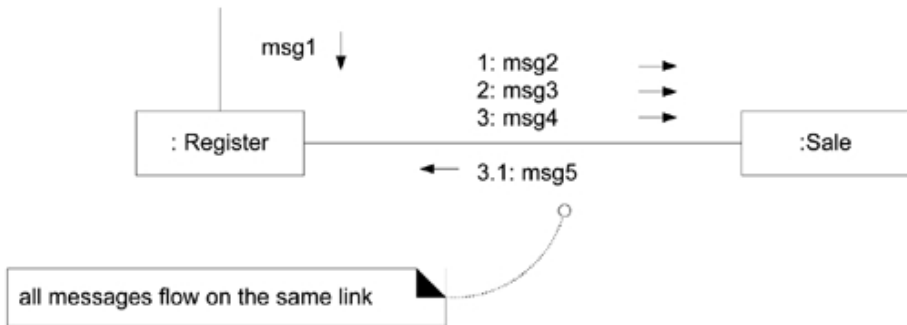


Diagramme de communication : exemple

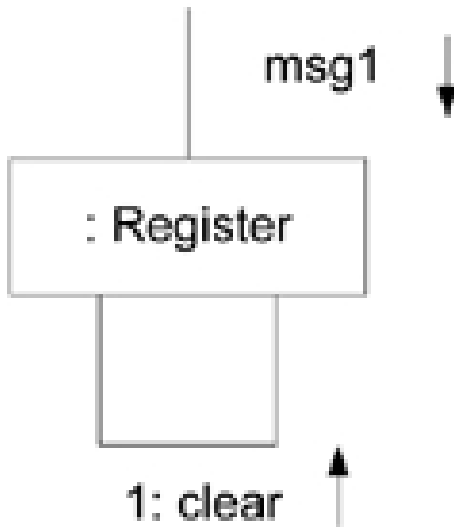
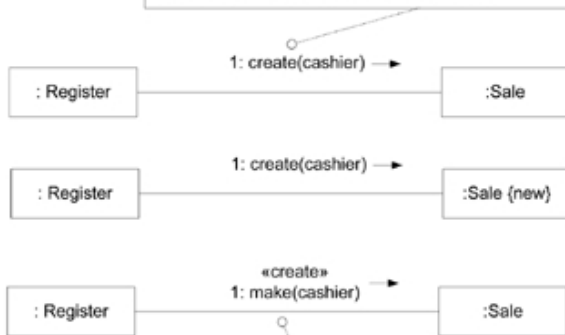


Diagramme de communication : exemple

Three ways to show creation in a communication diagram

create message, with optional initializing parameters. This will normally be interpreted as a constructor call.



if an unobvious creation message name is used, the message may be stereotyped for clarity

Diagramme de communication : exemple

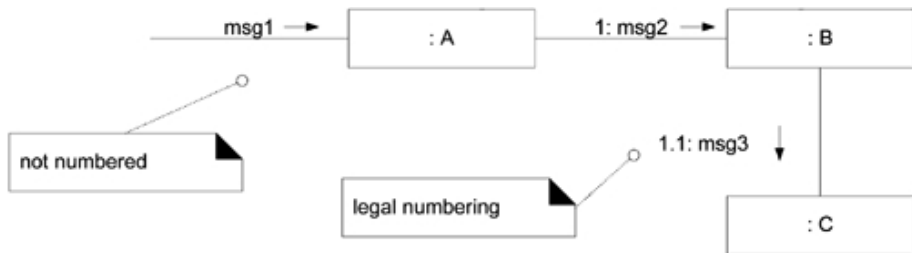


Diagramme de communication : exemple

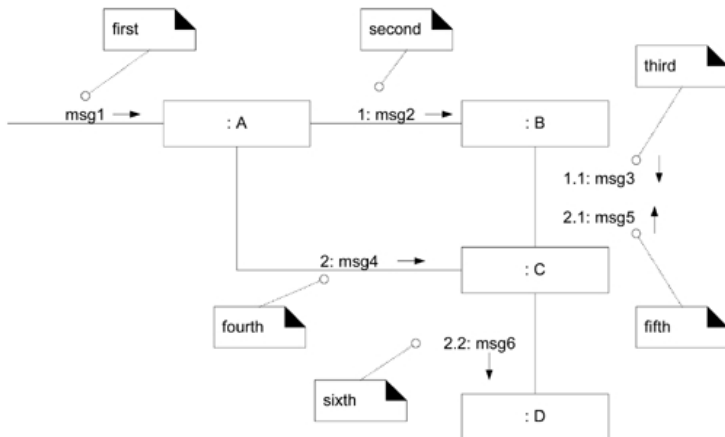


Diagramme de communication : exemple

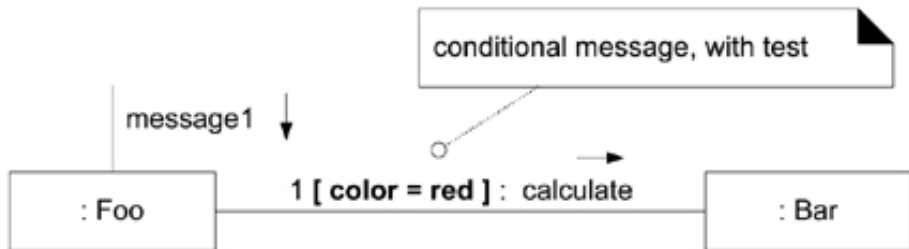


Diagramme de communication : exemple

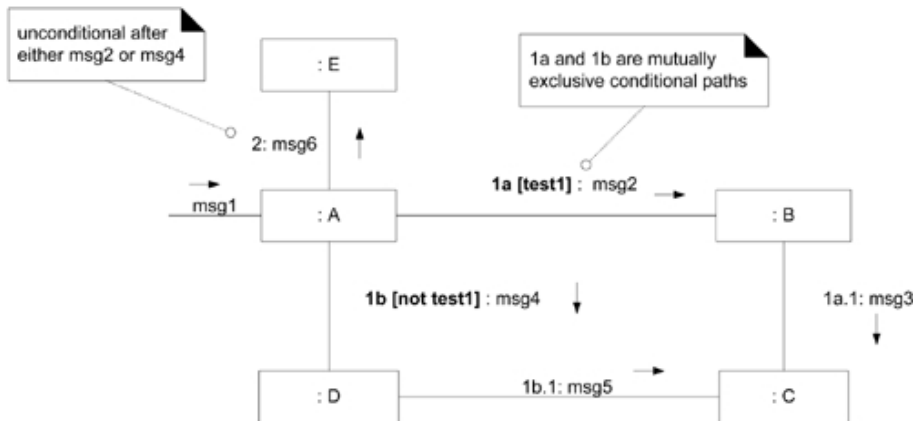


Diagramme de communication : exemple

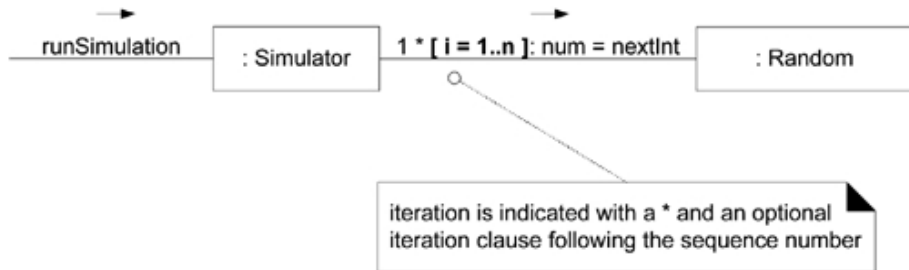


Diagramme de communication : exemple

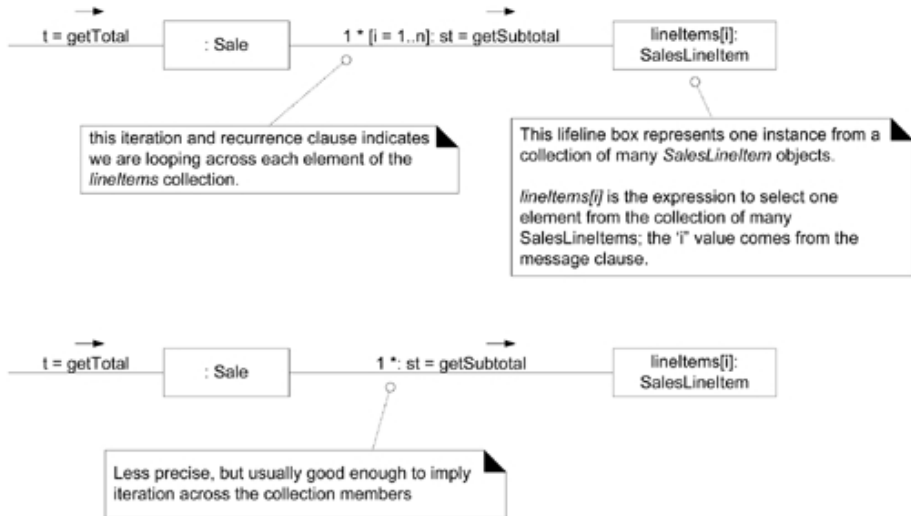


Diagramme de communication : exemple

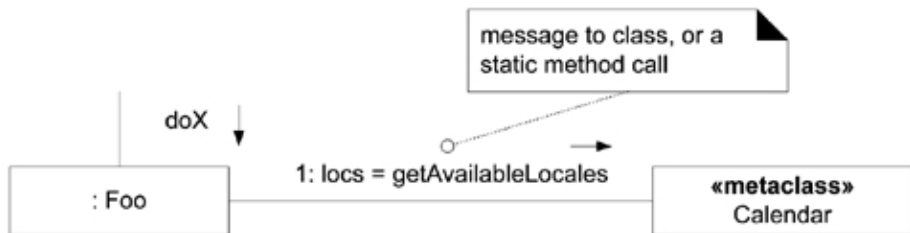
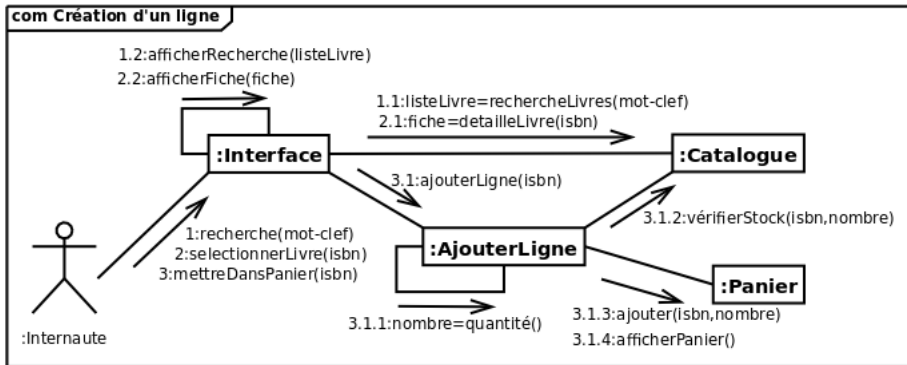
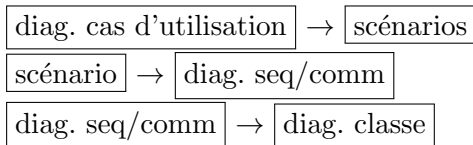


Diagramme de communication : exemple



Lien avec le diagramme de classes



Quand utiliser le modèle d'interaction ?

- Lorsque vous souhaitez visualiser le comportement de plusieurs objets dans un seul cas d'utilisation

Attention

- Si vous souhaitez visualisez le comportement d'un même objet dans plusieurs cas d'utilisation : utiliser le **diagramme d'états-transitions**
- Si vous souhaitez visualisez un comportement réparti sur plusieurs cas d'utilisation : utiliser le **diagramme d'activité**

Programmation Orientée Objet

— UML —

Cédric Buche

École Nationale d'Ingénieurs de Brest (ENIB)

20 novembre 2013

